



Sensor Data Evaluation From Industrial Wireless Sensor Network

Ahmad Ashraf Zargar, Raj Maharjan,
Shreya Jindal and Malik Zeeshan Ahmad

July 16, 2025

The study consists of analyzing sensor data provided from an Industrial Wireless Sensor Network (IWSN). Each sensor node is equipped with three different sensors: a Sensirion SCD30, a Sensirion SPS30, and a Bosch BME680. The motivation behind the study was to have data-driven methods for visualization, data cleaning, disruption / irregularities analysis with different granularity at node and network levels, classification / clustering methods and sensor behavior / replacement strategy. The project focuses on creating a framework which users can use to do through analysis for any dataset consisting of these sensor, which could be placed in different industrial settings. The framework allows the user to visualize the data, perform data cleaning, find different disturbances and irregularities on node level, perform data-driven process clustering, and have a sensor feature cross-analysis for replacement of sensor with redundant features.

1 Introduction

In modern industrial settings, sensor networks are becoming an increasingly essential part of environmental monitoring and operational intelligence. Industrial Wireless Sensor Networks (IWSNs) deploy sensor nodes throughout a set facility or room to track environmental parameters including but not limited to humidity, temperature, gas resistance, pressure and more. The increasing volume and complexity of the sensor data collected requires a robust analytical framework made with streamlit [1] that can help the user extract insights and make data-driven decisions regarding the facility moving forwards.

This projects is about the development of a comprehensive framework that is built specifically to analyze multi-sensor data collected from IWSNs. Each node has 3 sensors - Sensirion SCD30, Sensirion SPS30, and Bosch BME680. The framework built is designed to be flexible and scalable, to allow the user to visualize patterns, find irregularities and anomalies, make decisions supported by data and more.

Our approach combines statistical techniques, machine learning models and clustering methods to interpret and visualize patterns across the network and at the node level. It has data inspection, anomaly detection, process classification, cross-sensor and nodal redundancy evaluation, which combined can help the user make decisions on sensor placement, find malfunctioning nodes and sensors, and more. It also has a user-friendly interface built using Streamlit with modular capabilities, enabling the user to adjust, modify and scale the framework as necessary. The database attached to the framework can also be easily changed without needing to adjust much in the backend of the framework, making it easy to use for multiple different IWSNs.

2 Problem Statements

Industrial Wireless Sensor Networks continually produce large volumes of complex raw data. Without tools and a framework to analyze and clean the data, the data will be underutilized. The key challenges faced are as follows.

2.1 Data Quality Issues

Sensor data often has missing values, outliers and synchronization problems, which can in turn prevent trends from being identified, along with lead to incorrect interpretations or decisions made if not addressed systematically.

2.2 Fault Detection and Outliers

It is difficult to identify disruptions in the data, issues with sensors and remove faulty and erroneous readings without a detection framework. Manually inspecting the data is not feasible with such a large database, especially with more data being added every day.

2.3 Pattern Recognition

There are repeated patterns that will be revealed over time and with increasing amounts of data collected. It is not possible to identify these patterns from the raw data without an automated process collecting, cleaning and analyzing the data as it is added to the database.

2.4 Sensor and Node Redundancy and Optimization

Each node has sensors that record same environmental features. For example, temperature is recorded by more than 1 sensor. Similarly, each node has the same 3 sensors recording the data. Without any further analysis, unnecessary sensors or nodes may be being used, increasing the costs through purchasing and maintaining redundant sensors and nodes.

3 Data Visualization

Data Visualization is one of the most important aspect for any data-driven project. Data Visualization provides the readers with the basic underlying structure of the data which could be exploited in the later stage of the analysis. Data Visualization is also important to finding various trends, outliers and irregularities in the data which the users might want to either analyze or remove from the dataset to have a more refined analysis in terms of using statistical learning tools.

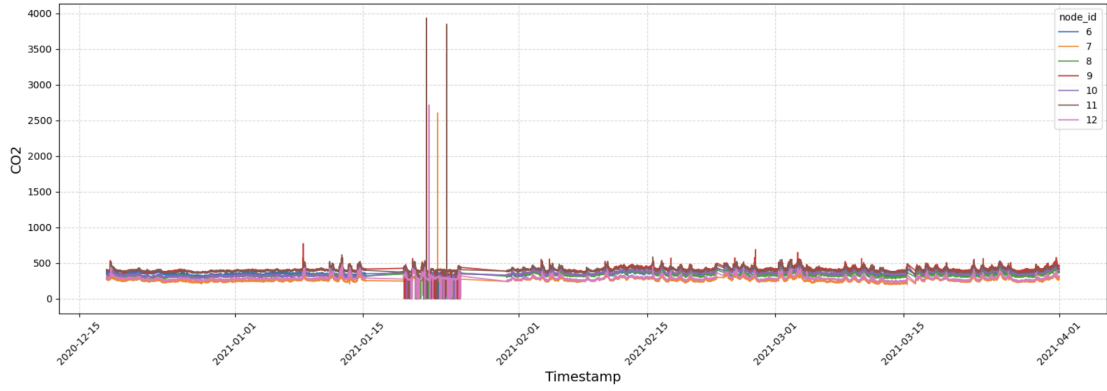


Figure 1: Sensor-SCD30 CO₂ readings over an selected period from 12.2020 - 04.2021

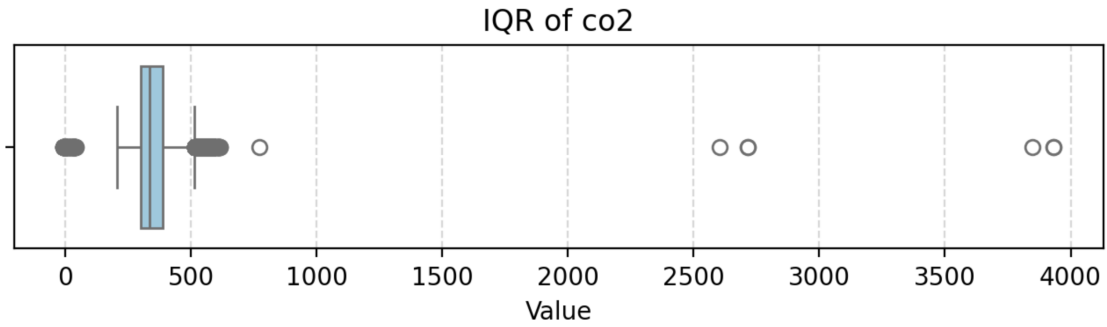


Figure 2: IQR Visualization for Sensor-SCD30 CO₂ with outliers [01.2021 - 02.2021]

Knowing the importance of data visualization our framework includes a module where the users can easily access their underlying data and start with basic visualization. The framework allows the users to select the sensor table which interest them for the visualization with an option to filter out the time period of interest. An example of visualization with different granularity level for a period with high outliers / irregularities could be seen from Figure 1, which also presents the motivation for section 4.

Another important aspect to understand the data is to visualize the quartile ranges present in the data; an example is seen in Figure 2 where the IQR for CO₂ reading shows the left-skewness of a period which has a lot of outliers, while Figure 3 refers to a period with minimal outliers or more normal processing going on which releases uneven distribution of CO₂ in the industrial environment.

Visualization also provides some insights for section 5, where Figure 4 provides the insights that not all sensor nodes are recording the same or equal amounts of reading, although for the selected period it seems minimal. Reexamining Figure 1, would also provides insights that the sensor network has missing gaps with respect to the timestamps, which are also prominent in 5.

To provide more context in what the data visualization module in our framework does:

- Time-Series Trend Analysis: visualization of sensor reading with different granularity, i.e., to either have a plot which includes different color gradients based on node id (Figure 1), or to have individual plots for each node, to understand node-level behavior (Figure 5).
- Correlative Feature Analysis: enabling selection of two features to be compared over the

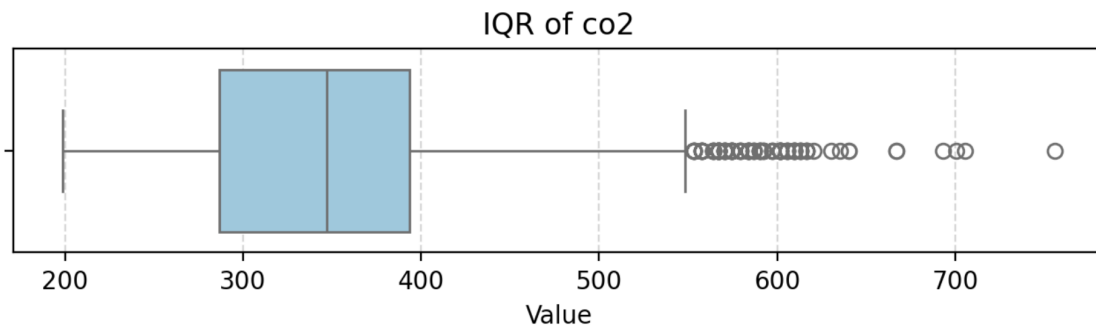


Figure 3: IQR Visualization for Sensor-SCD30 CO₂ with normal processing [04.2021 - 05.2021]

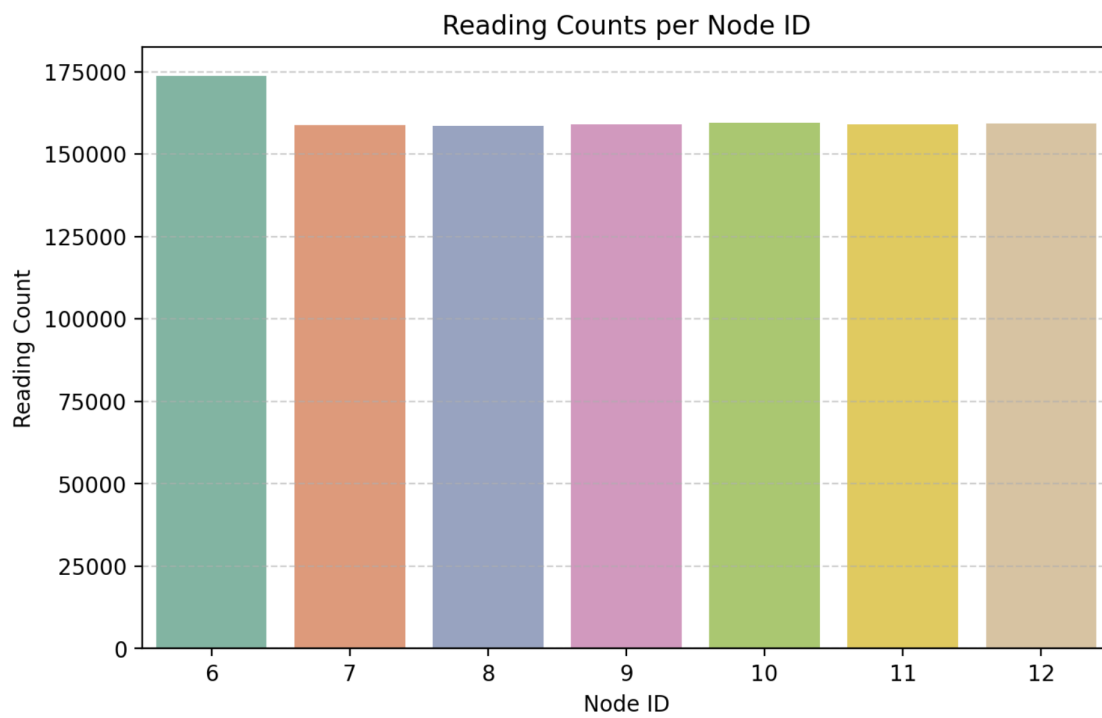


Figure 4: Sensor-SCD30 per node recording analysis [01.2021 - 02.2021]

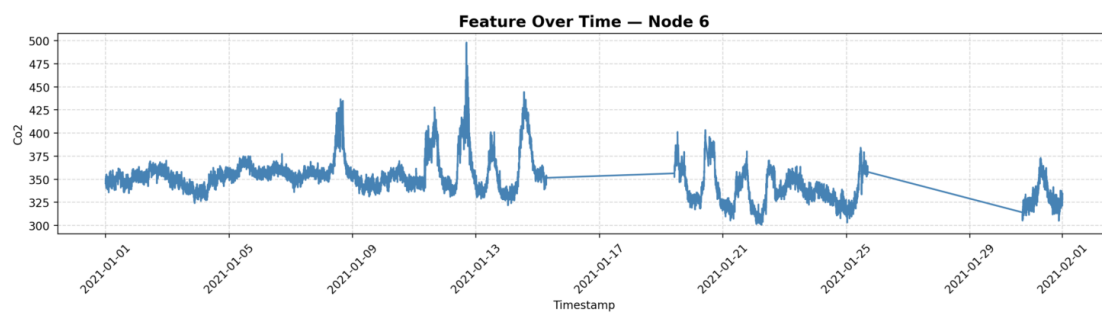


Figure 5: Sensor-SCD30 CO₂ readings for NodeId 6

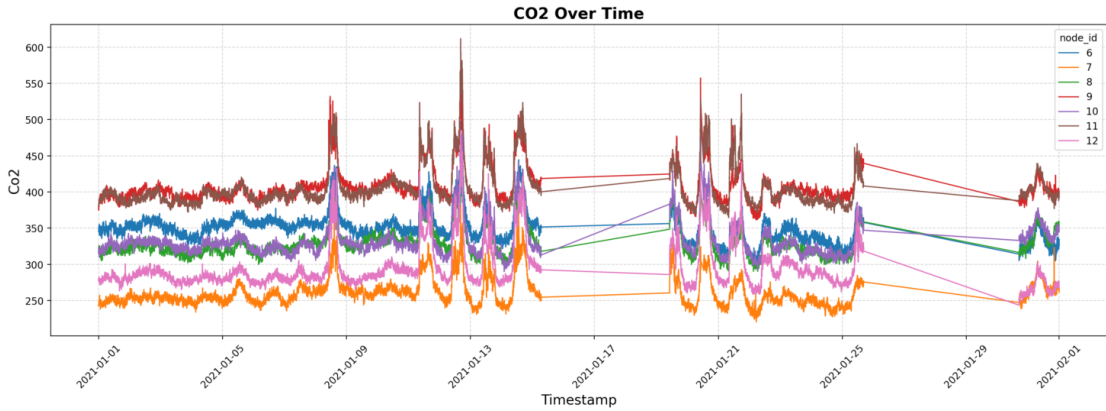


Figure 6: Sensor-SCD30 CO₂ with outliers removed using hardcoded bounds [01.2021-02.2021]

same time period, helping to uncover daily operational patterns or intra-sensor feature relationships.

- **Hourly Pattern Analysis:** for any selected feature, the user can have daily plots of hourly average values recorded with the granularity of the node. This could be helpful in trend analysis, anomalies, or shifts.
- **Statistical Summaries and Distributions:** provides statistical summary of the dataset, Boxplots / Interquartile Range (IQR) Visualization for variability, skewness, and outliers

4 Data Cleaning

Even though Data Visualization is the most critical part of any analysis, it still mostly follows a clear structure. While on the other hand after data visualization the most extensive and semantically challenging task of cleaning the data comes into play. After an extensive analysis of the data, different bounds of outliers can be determined, or a simpler method of only having bounds in which the sensor operates could be considered.

Using bounds of sensor, based on the sensor configuration sheet is easier and does not require much semantical understanding, but the problem with such an approach is in terms of losing higher granularity of data, which could hold various insights which the user or the operator are interested. Therefore, an appropriate strategy always depends on the use case and the computation or semantic cost required. Figure 6 presents the cleaned view of the data which could be compared with respect to Figure 1 where the outliers were still present in the data.

Starting with observation from data visualization, it was noticed that in the data, we have periods when the sensor network has nodes which according to the deployment should not be considered for data analysis, it could be maintenance periods or some other factors. This could be checked from the Sensor-SCD30 data for the period 01.2021 - 02.2021, when a total of 13 nodes are recording unique reading. While from the deployment only 7 unique nodes should be present in the industrial setting, for such cases the framework gives the users the ability to select nodes which should be present in the data, and returns the cleaned version of the data consisting only of the nodes which the users have selected. An example can be seen from Figure 7, where according to the deployment sheet only nodes [6, 7, 8, 9, 10, 11, 12] should be recording the data.

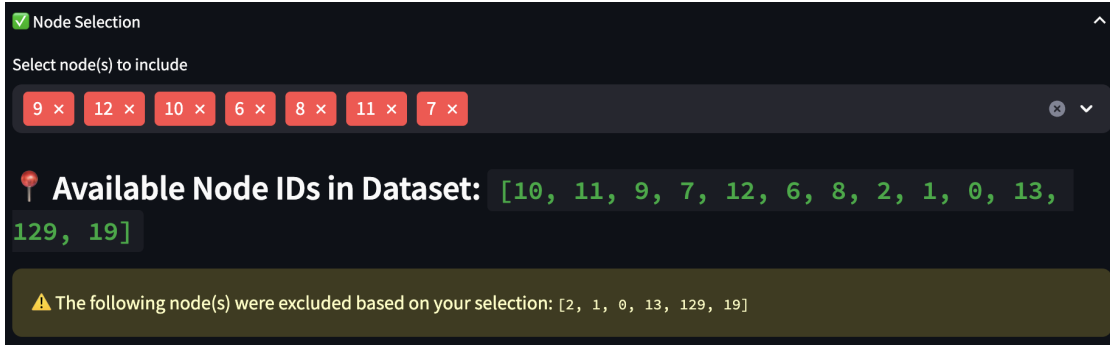


Figure 7: Sensor-SCD30 unique node ids present in the data

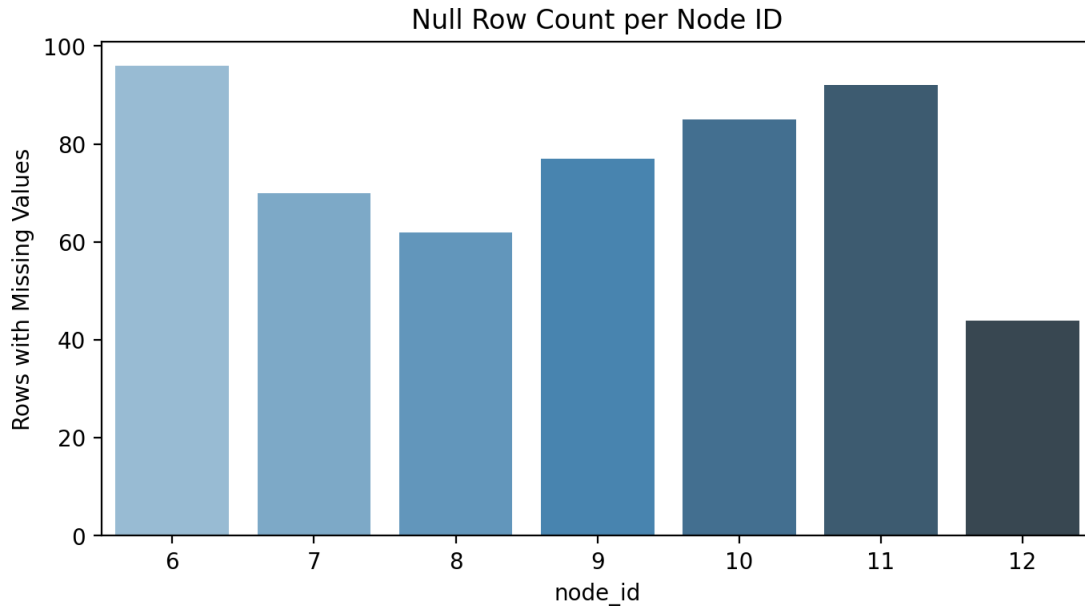


Figure 8: Bar plot for null values reported by Sensor-SCD30 [01.2021 - 02.2021]

Similarly, null values are removed from any of the feature columns, as they are not supported by most of the machine learning model or statistical tools. The framework also provides the user with an overview of all the null values in the data, and also presents it on node ID granularity to verify if some sensors are malfunctioning or are not calibrated. An example is also seen in Figure 8.

Moving towards the cleaning of outliers from the data, where the actual human semantics comes into play. For our framework, we have provided the users with three different methods of outlier data cleaning, which are presented in the subsequent sections.

4.1 Hard-coded Outlier Bounds

The system can utilize bounds-based outlier detection where sensor readings are compared to hard-coded thresholds for each variable like temperature, humidity, and CO_2 . Such limits are drawn from manufacturer specifications, domain knowledge, or historic patterns. For instance, Sensor-SCD30 are expected to report temperatures from 0°C - 50°C and CO_2 levels from 400 to

10.000 ppm during typical operations. Each data set value that is within the invalid range is treated as an outlier and removed along with the outliers in the cleaned data set.

The system offers a straightforward and easy-to-comprehend rule-based filter for a well-defined and static environment. More importantly, the system permits users to set limits for each sensor type or variable — therefore allowing for configurations based on the actual needs of the specific deployment scenarios. As an example, the user deploying a sensor in a cold storage facility may change the valid temperature range to 0°C–10°C. Such bounds which are not hard-coded into the system can be provided to the framework and used instead of the default ranges, thus enabling the framework to be used in many situations.

From conducting extensive data analysis, the bounds found for the dataset provided are in Table 1. These bounds specify the reading which are not considered as outliers and which are considered.

Table 1: Hard-coded bounds for outlier removal based on sensor type. Values outside these bounds are considered outliers and removed from the dataset.

Sensor Type	Variable	Valid Range
SCD30	CO ₂ (ppm)	[171.28, 939.69]
	Temperature (°C)	[16.6, 32]
	Humidity (%)	[14.72, 65.39]
BME680	Temperature_bme (°C)	[13, 33.61]
	Humidity_bme (%)	[10.45, 61.45]
	Pressure (Pa)	[61521, 103777]
	Gas Resistance (Ohm)	[3000, 5430598]

These bounds are also useful in creating machine learning models, which could have online learning and can be deployed on the sensors itself to flag the reading or sensor nodes which are reporting such values. A simpler SVM based classifier model is also employed by our framework. One of the disadvantages of this approach would be losing the granularity. Similarly, defining the bounds also requires a fair bit of semantic data analysis.

4.2 SVM-Based Outlier Removal

This approach is dependent on the bounds specified in the previous section, which are used to mark the data entries as outliers where the row has features outside those bounds. Such an approach could be used for developing lightweight models which could be deployed on sensor networks with online learning mechanisms where the decision boundary is dynamic and also takes into account the trend shifts, temperature variations and other cyclic features. This model is considered as a starting point, though it suffers from the same problem as the method before, and the model created works when both Sensor-BME680 and Sensor-SCD30 are considered together. An example of the decision boundary created by the SVM model is shown in Figure 9, considering the feature dimension of CO₂ and temperature.

Other machine learning and deep learning models were also considered, but such models faced the problem of False outlier detection. Where an higher percentage of data was classified as outliers, while they were actually correct readings. Online pre-trained models such as PyOdds [2], also suffers from higher false outlier detection. Similarly neural networks with different hidden layers were considered which also suffered from the same fate. Figure 10 showcases the

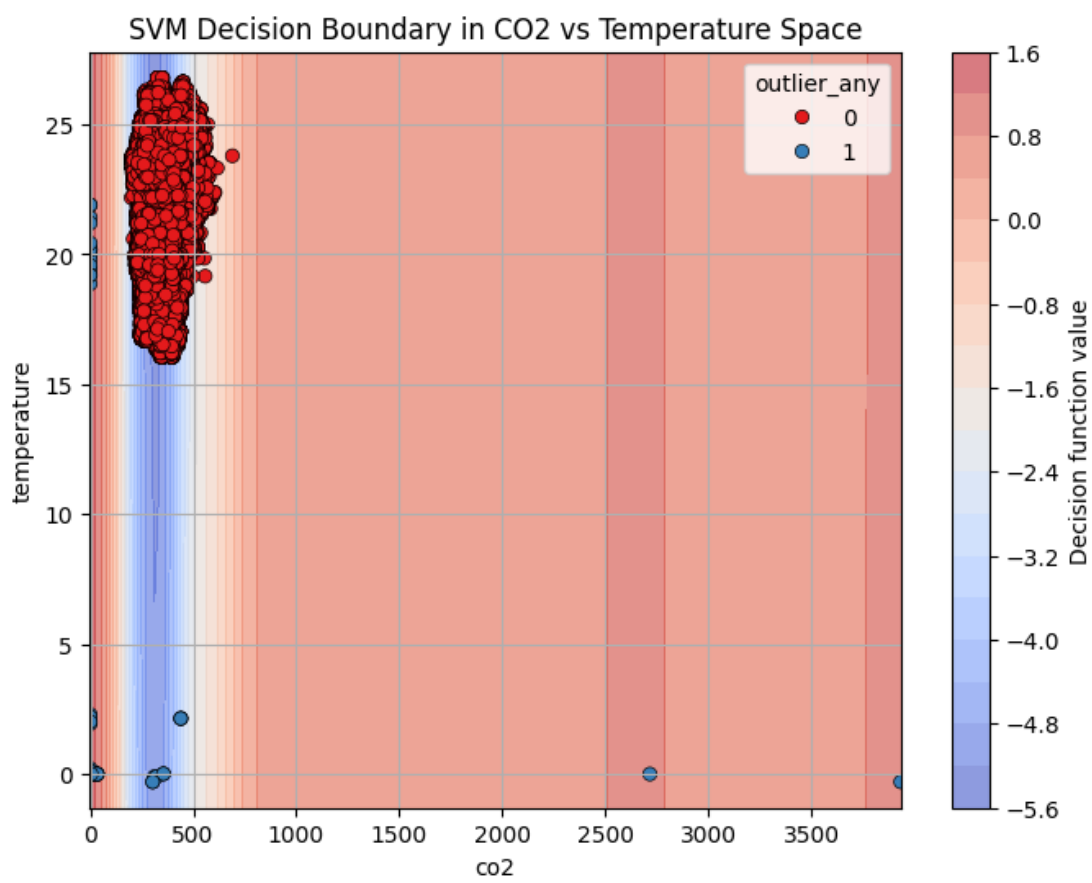


Figure 9: Multi-Sensor SCD30-BME680 SVM detection method

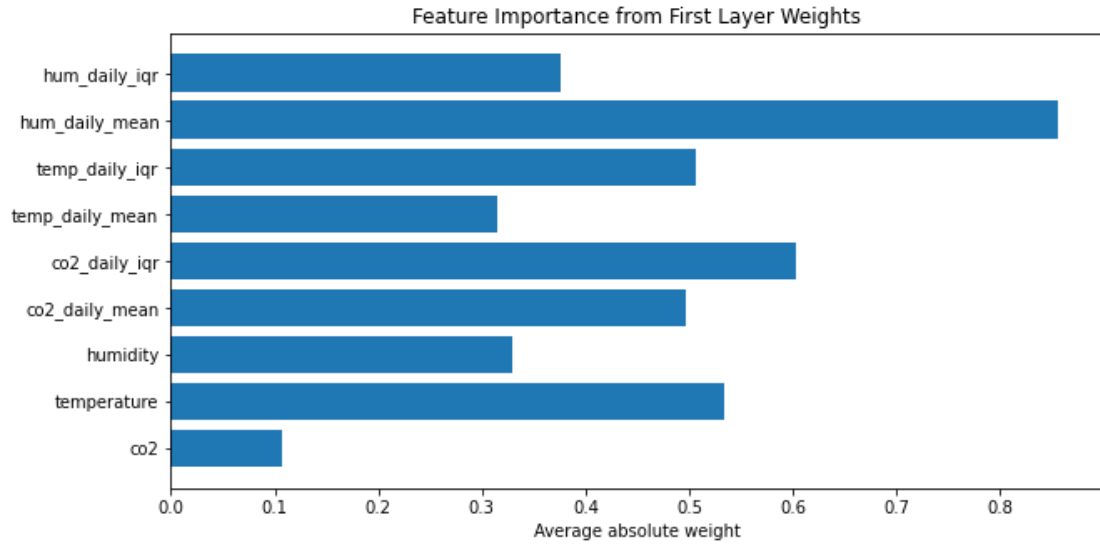


Figure 10: Neural Network feature importance, trained on SCD30-BME680.

importance of features according to the neural network. In the study we've also considered using different SVM kernel, but due the curse of dimensionality only RBF kernel was able to have an acceptable runtime, while linear and poly kernels where not able to produce any results.

4.3 Inter-Quartile Range Outlier Filtering

IQR filtering according to our analysis is more diverse in handling data with different semantics. Our framework incorporates this as a recommended method because of its ability to dynamically adjust the bounds based on user input. Such an IQR method does not take into account any biases and just uses simple statistical tools to filter any data point which is not in the majority share of the distribution.

In our framework, we have considered a weighted IQR method where the user can select the upper quartile and lower quartile range for outlier cleaning, which is weighted to only remove values which are extremely deviated with respect to the IQR whiskers. IQR filtering enables the framework to be dataset independent, and gives the user the ability to dynamically change the outliers and remove them. While providing cleaned and outlier version of data which the users can download for further analysis. Table 2 presents the outliers removed from the dataset over a period with high outlier count, where the readers can see from the temperature reading, that such values are not expected. Similarly for CO_2 and humidity the statistics of outliers are presented in the Table.

5 Disruption / Irregularity Analysis

Disruption analysis is another critical component of any IoT setups. For both deployed environments and service-providers, understanding disruptions is essential to determine the efficiency and robustness of the network. Disruptions can occur at various levels, be they at a node level, a sensor level, or even across the entire network.

Table 2: Statistics Summary for Sensor-SCD30 Outlier readings (214 samples) [01.2021 - 02.2021]

Statistic	CO ₂ (ppm)	Temperature (°C)	Humidity (%r.H.)
Count	214	214	214
Mean	166.15	2029.64	68.55
Min	-4.80	-4.50	-4.10
25%	-0.01	-0.01	0
50%	1.95	0	0
75%	38.45	0.01	0
Max	3932.90	215622.34	8700.22
Std Dev	558.97	20401.72	657

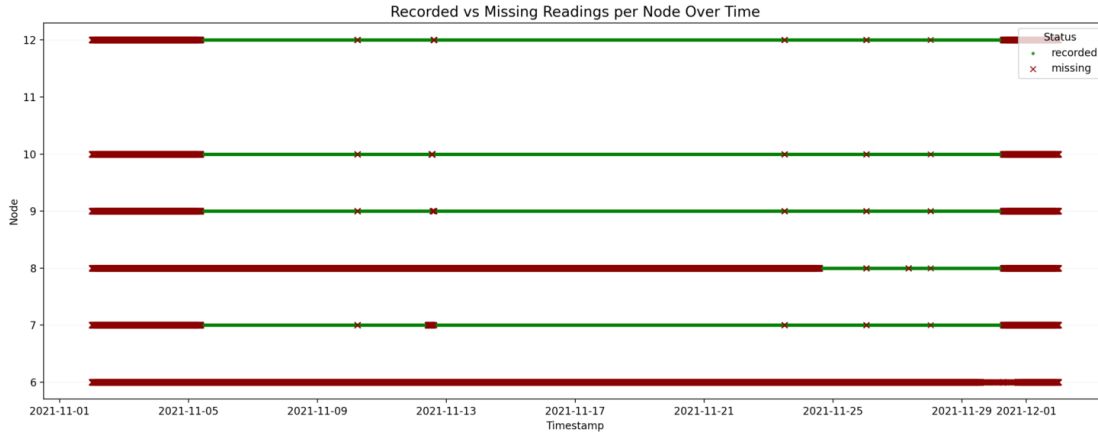


Figure 11: Recorded and Missing data for Sensor-BME680 [11.2021 - 12.2021]

By examining the intervals set for data recording and identifying when no recordings are held, we can measure both network up-time and down-time.

These analyses give us a clear picture of how well our IoT setup is working in industrial environments. By looking at the metrics, we can figure out when the network got disrupted and how it affected the system. We can also spot if there are any repeating disruptions in the network.

Such examples of such sensor node disruptions can already be seen from figure 1, but to have a much broader view of such disruptions, we have included them into the framework. Where the expected sensor recording size is compared to what actually is present in our data, for the given dataset, the expected reading interval is around 10 seconds, while in the framework, we have provided a way to modify this interval dynamically, which could be used for the analysis of different sensor deployments with different semantics. We have also considered clock drifts in the sensor network. To facilitate such scenarios, we have also introduced jitter of 10 seconds (which is dynamically adjustable), which means at least 1 reading should be recorded into the data by the sensor with node ID granularity. Figure 11 presents a highly disruptive month [11.2021 - 12.2021] for Sensor-BME680.

Now, using the same interval information we could also provide an analysis for up-time vs. down-time, i.e. the number of recording in the data vs. the expected one, Figure 12 showcases this analyses for the same time period as above with the granularity of node ID.

Our framework also incorporates thresholding mechanisms which could dynamically assign

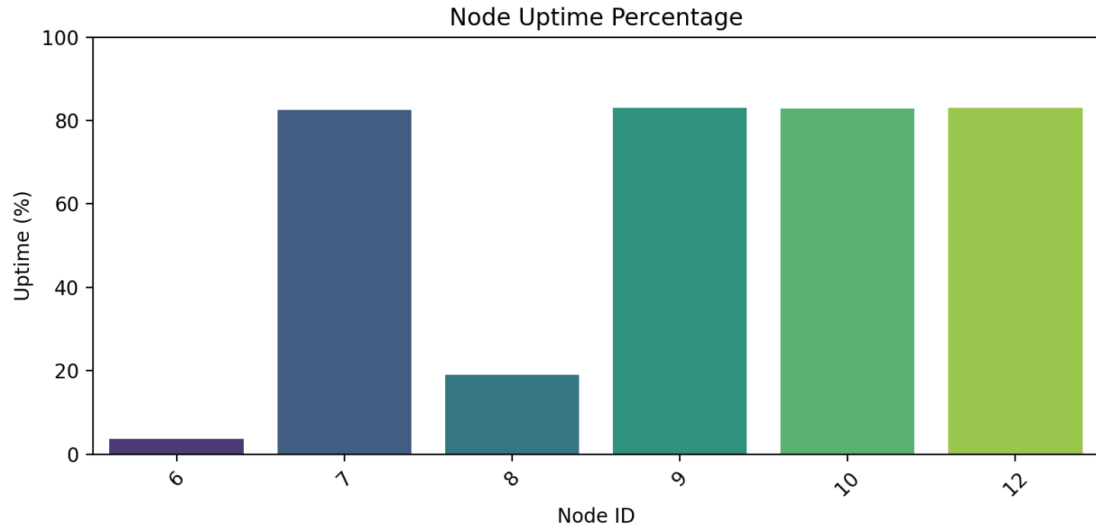


Figure 12: Uptime analysis for Sensor-BME680 [11.2021 - 12.2021]

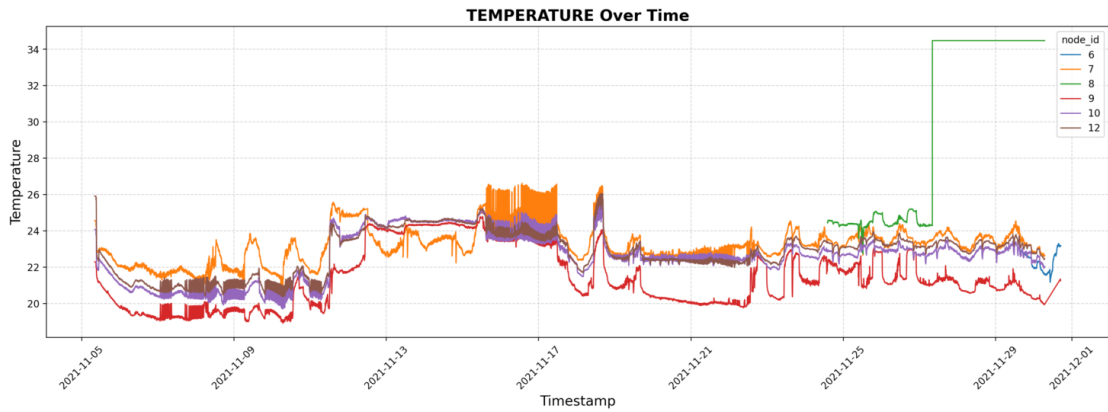


Figure 13: Sensor-BME680 with NodeId 8 recording faulty temperature reading [11.2021 - 12.2021]

such disruptions into four different groups based on the time period; this section is fully customizable where the user can assign different thresholding for connivance.

Along the same lines, the framework consists of logic, where based on user input, the method would show disruptions which are global or a shortage of the entire network. In other words, where all sensor nodes experienced disruptions simultaneously.

Another key trend which we had noticed from Section 3, was that some sensor nodes could be stuck in a faulty period, where the sensor nodes are recording the same feature or reading for an extensive amount of time. An example can be seen from Figure 13, where Sensor-BME680 Node ID 8 for the later part of the month is recording faulty readings.

For such scenarios, we have again employed a lightweight model; in this case, a simple logistic regression model is employed. Trained for a single month of data with such trends, the model generalized well enough to classify such readings as irregular / faulty readings. This could be extremely important as such lightweight models could be deployed again to the sensor nodes, which could flag the readings and, after a certain threshold, trigger a required maintenance or calibration mode.

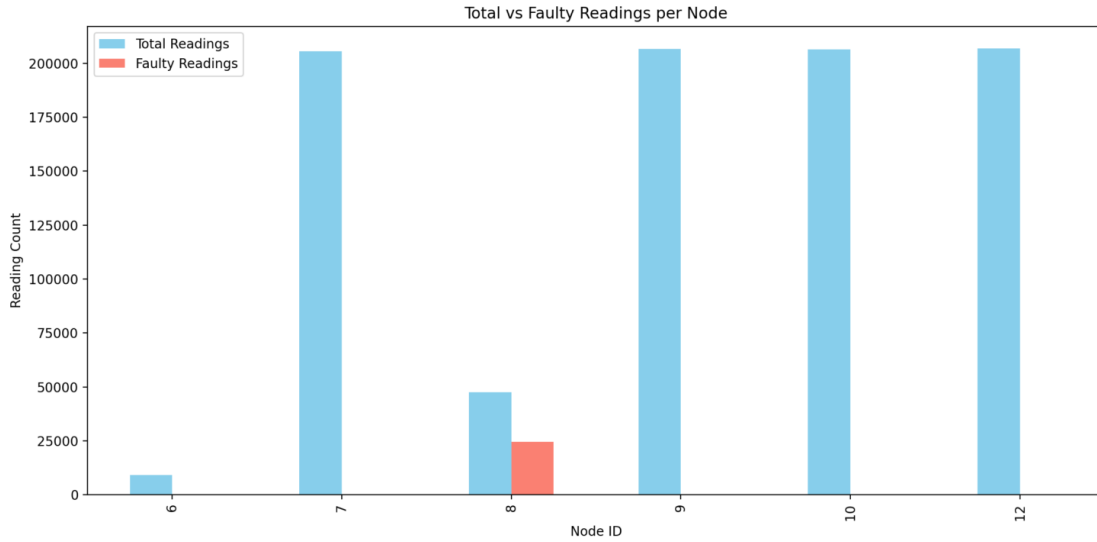


Figure 14: Faulty vs Total Readings per Node for BME680 [11.2021 - 12.2021], with node ID 8 reporting faulty readings

An example of such triggering mechanisms is already employed in the framework as well and is shown in Figure 14, which showcases the amount of faulty readings which are recorded by each sensor. Where Table 3 and Table 3 present the difference in the recorded values, which can be noticed for temperature and pressure, where the standard deviation between the recordings in Table 4 is 0. Such statistical results are also present in our framework.

Table 3: Summary statistics for normal Sensor-BME680 readings (857,585 samples).

Statistic	Humidity (%r.H.)	Temperature (°C)	Gas Resistance (Ohm)	Pressure (Pa)
Count	857585	857585	857585	857585
Mean	32.35	22.55	224097.81	100764.02
Std Dev	4.87	1.48	101765.37	1093.30
Min	22.25	18.91	77526	97713
25%	28.56	21.42	150765	100190
50%	31.92	22.72	199651	101165
75%	36.25	23.62	272051	101604
Max	52.76	26.62	623794	102032

6 Intelligent Sensor State Analysis System

6.1 Introduction and Problem Statement

Understanding complex sensor data patterns in real-time has become increasingly important as monitoring systems produce thousands of data points daily. Traditional methods rely on experts to analyze multi-dimensional sensor streams manually. This process is time-consuming, costly, and practically impossible to sustain across multiple facilities and operational settings.

Table 4: Summary statistics for faulty BME sensor readings (24,567 samples).

Statistic	Humidity (%r.H.)	Temperature (°C)	Gas Resistance (Ohm)	Pressure (Pa)
Count	24567	24567	24567	24567
Mean	25.23	34.47	245985.34	63749
Std Dev	1.11	0.00	22673.26	0.00
Min	23.09	34.47	208103	63749
25%	24.60	34.47	231998	63749
50%	25.32	34.47	245304	63749
75%	26.07	34.47	260228	63749
Max	27.40	34.47	301432	63749

Our intelligent sensor state analysis system bridges this gap by converting continuous streams of multi-dimensional sensor data into valuable insights about operational states and processes. Instead of relying on manual interpretation or pre-labeled training data, we developed an unsupervised machine learning method that automatically identifies operational patterns directly from the sensor measurements.

The system addresses three main challenges organizations face today. First, pattern recognition becomes complex in multi-sensor environments that generate vast amounts of data which traditional analysis methods cannot effectively process. Second, real-time state classification is essential for enabling proactive responses instead of reactive troubleshooting. Third, distinguishing between active operational periods and baseline conditions is difficult without prior knowledge of how the system behaves.

What sets our approach apart is that it requires no manual labeling or specialized knowledge. We let patterns emerge naturally from the sensor data, allowing the data to tell its own story. This unsupervised approach means the system can adapt to various operational environments, discovering operational states based on actual conditions instead of preconceived notions about when things should happen.

The expected results include automated process detection that determines when significant activities occur, state classification that categorizes operational modes based on sensor data, pattern discovery that reveals temporal rhythms in operational schedules, and real-time monitoring abilities that provide immediate classification of current operational states.

6.2 Data Foundation and Multi-Sensor Integration

Our analysis system combines data from three specialized sensor platforms to create a comprehensive environmental monitoring ecosystem. The foundation comprises fifteen distinct sensor features that capture various aspects of environmental and operational conditions, providing the dimensional richness needed for effective pattern discovery across different operational states.

The BME680 environmental sensor serves as our primary monitoring platform. This sensor provides the fundamental environmental state that helps us understand baseline conditions and process-related air quality changes.

The SCD30 CO₂ sensor measures carbon dioxide levels. While this represents just one feature in our analysis, CO₂ concentration often serves as our primary indicator for occupancy patterns, ventilation effectiveness, and process activity detection because it responds directly to human presence and operational activities.

The SPS30 particulate matter sensor provides detailed air quality data through ten different measurements. It captures mass concentrations for PM1, PM2.5, PM4, and PM10. Number concentrations across size ranges in particles per cubic centimeter, and average particle diameter in micrometers. This particulate analysis enables the detection of process emissions, equipment operations, and fine-scale air quality changes that other sensors might miss.

The robust data infrastructure supporting this multi-sensor ecosystem relies on a SQLite backend, which offers dependable, serverless database storage for continuous data streams. The normalized schema maintains separate sensor tables connected by measurement timestamps, which supports a scalable design that adjusts to different sensor configurations and data volumes. This ability of flexibility enables configurable analysis periods from single days to full years, with adaptive time filtering that can focus on working hours or exclude weekends based on operational requirements.

Our data preprocessing pipeline ensures high-quality input for pattern recognition through a four-stage quality assurance process. Missing value handling automatically identifies and removes incomplete measurements, typically resulting in only 2 to 5 percent data loss during sensor maintenance. Statistical outlier detection uses z-score filtering with a threshold of absolute z greater than or equal to 3, removing approximately 1 to 3 percent of anomalous readings from sensor calibration events. Feature standardization uses StandardScaler normalization to achieve zero mean and unit variance across all sensors, preventing features with large ranges, like gas resistance, from overshadowing those with smaller ranges, like humidity percentages. Finally, comprehensive data validation checks integrity and verifies sensor functionality, typically retaining over 95 percent of raw data after the complete pipeline.

6.3 Advanced Clustering and Pattern Discovery Methods

The heart of our pattern discovery relies on an advanced clustering method based on a simple but effective philosophy: let the data speak for itself. We apply K-means clustering purely to sensor values without any temporal bias, allowing operational states to emerge from the relationships between measurements instead of relying on temporal assumptions.

As shown in Figure 15, our clustering results illustrate how sensor measurements naturally group into distinct operational modes in the principal component space. Each point represents a single measurement colored by its cluster assignment. The clear separation visible in both PC1 versus PC2 and PC1 versus PC3 views confirms that our fifteen-dimensional sensor measurements form distinct operational states. This sensor-based approach makes sure that the operational states we identify reflect actual physical conditions captured by our sensors.

The dimensionality reduction process transforms our fifteen-dimensional sensor space into interpretable visualizations through Principal Component Analysis. Figure 16 demonstrates that we typically capture 80 to 90 percent of total variance with just three components, with PC1 typically accounting for about 50 percent of the variance and representing the primary operational differences. The cumulative variance shown in Figure 17 confirms we exceed the 90 percent threshold by the third component, meaning we retain most essential information while making the data visualizable in two- and three-dimensional space.

Our algorithm comparison study highlights the practical advantages of K-means over other clustering methods. While DBSCAN can identify complex cluster shapes, our analysis shows that K-means provides the best balance of speed and interpretability for real-time operational monitoring. As demonstrated in Figures 18 and 19, K-means produces clear cluster definitions in under one second, while DBSCAN takes significantly longer, making K-means more suitable

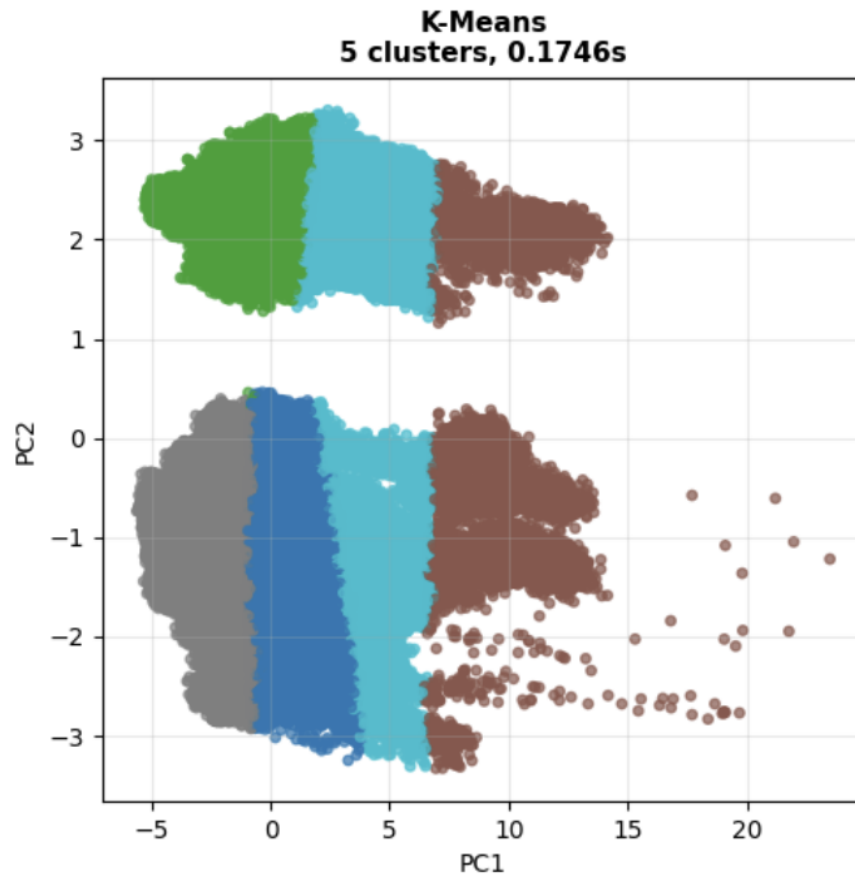


Figure 15: K-means clustering results showing how sensor measurements naturally group into distinct operational modes in principal component space

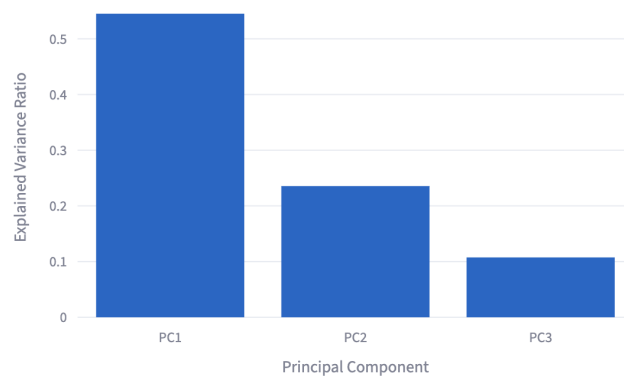


Figure 16: Principal Component Analysis variance explanation showing the contribution of each component

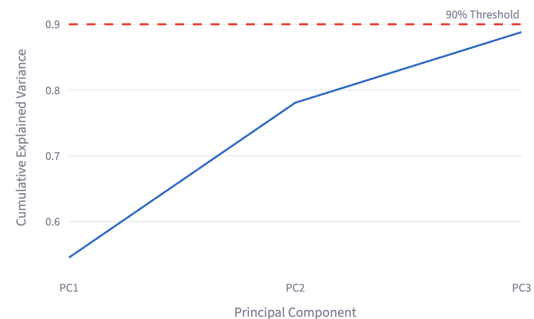


Figure 17: Caption

for continuous sensor stream processing.

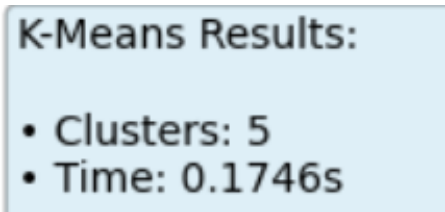


Figure 18: K-means clustering performance showing fast execution time

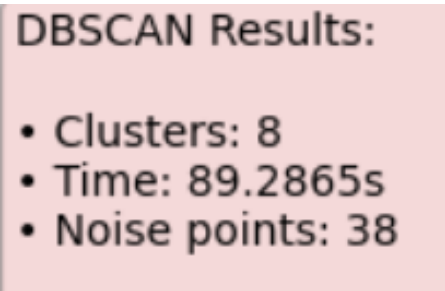


Figure 19: DBSCAN clustering performance comparison showing longer execution time

The preprocessing pipeline supports robust algorithm performance through comprehensive quality control. Missing value removal manages sensor maintenance periods and connectivity issues that typically affect 2 to 5 percent of measurements. Statistical z-score filtering with absolute z greater than or equal to 3 eliminates sensor calibration events and outlier readings while preserving legitimate process variations. StandardScaler normalization prevents range bias among sensors by ensuring equal influence of features regardless of measurement units, and thorough validation performs cross-sensor synchronization checks to uphold data integrity.

The clustering configuration provides operational flexibility through user-controlled parameters. Adjusting cluster counts from 2 to 12 allows analysis granularity to match operational complexity. Two clusters provide basic active versus inactive classification, while 4 to 6 clusters offer typical operational state detail, and 8 to 12 clusters permit finer process differentiation. Adaptive filtering options allow focusing on working hours, excluding weekends, and setting flexible date ranges for different operational contexts and analysis needs.

6.4 Temporal Analysis and Process Day Detection

One of the most remarkable aspects of our clustering approach is how purely sensor-based analysis naturally aligns with temporal operational patterns. Figure 20 shows how clusters follow hourly boundaries throughout a day without using any temporal features in the clustering algorithm. The purple, brown, and orange clusters dominate early morning and late evening hours, representing baseline environmental conditions when activity is minimal. In contrast, blue, red, and green clusters appear during working hours, each reflecting different operational activities with unique sensor signatures.

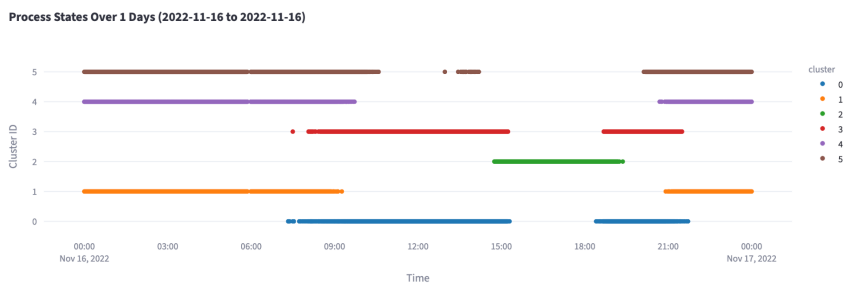


Figure 20: Single day temporal analysis showing cluster alignment with hourly boundaries and operational patterns

This temporal alignment beautifully validates our methodology, showing that the clusters correspond to real operational states that follow natural work patterns. Figure 21 expands this analysis across a full week, revealing which operational states repeat on different days and how patterns change between weekdays and weekends. Some clusters consistently appear on weekdays, indicating regular operational activities, while weekend periods show different cluster patterns with less variety and fewer active states.

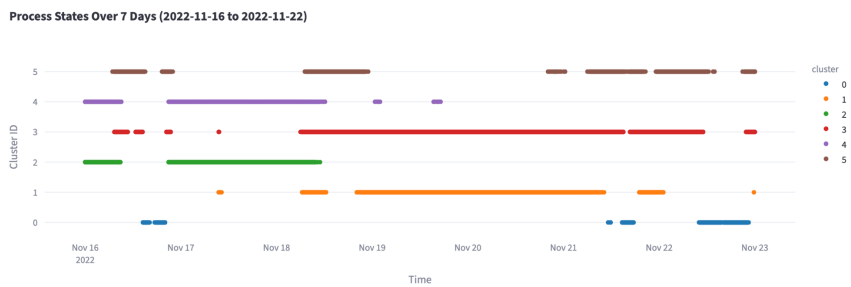


Figure 21: Seven-day temporal analysis revealing weekly operational patterns and state transitions

Our process day detection method provides another level of unsupervised analysis that automatically classifies entire days as either process days or baseline days. This approach uses CO2 levels during working hours as the primary indicator, applying the logic that higher working-hour CO2 levels usually indicate more human activity and operational processes. The system automatically assigns labels through K-means clustering at the daily level, then maps clusters based on CO2 signatures without needing manual labeling.

Figure 22 shows that among the analyzed days, the system typically identifies 60 to 70 percent as process days, representing weekday operational periods, while 30 to 40 percent

are classified as baseline days, covering weekends, holidays, and low-activity periods. This automated classification provides a high-level operational overview that can inform scheduling, resource planning, and highlight unusual activity patterns that deviate from expected operations.

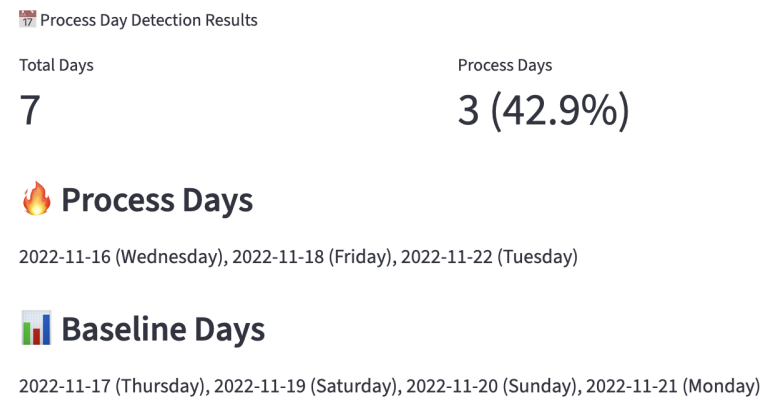


Figure 22: Process day detection results showing automatic classification of operational versus baseline days

The day-of-week analysis reveals fascinating insights about institutional work schedules discovered purely through sensor pattern analysis. Figure 23 shows weekends are largely dominated by baseline clusters, while weekdays showcase a full range of operational activities from baseline conditions to various active processes. Figure 24 presents the normalized distribution highlighting how certain days, including some Mondays, show minimal activity patterns similar to weekends, suggesting holidays or reduced operational schedules that the algorithm detects automatically.

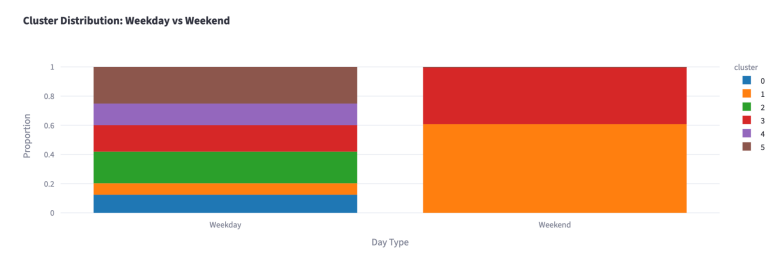


Figure 23: Weekday versus weekend operational pattern comparison showing distinct activity profiles

Weekly temporal pattern analysis demonstrates the system’s ability to identify recurring operational rhythms across different time scales. The observed temporal consistency across multiple weeks validates the stability of the patterns and confirms that the clustering methodology reliably identifies operational states that reflect actual facility usage patterns. This capability enables predictive scheduling, resource optimization, and automated detection of deviations from normal operational behavior.

6.5 Real-time Classification and Practical Applications

The system’s real-time classification capabilities bridge the gap between offline pattern discovery and live operational monitoring. Our interactive prediction interface, shown in Figure 25, allows

Normalized Cluster Distribution by Day of Week

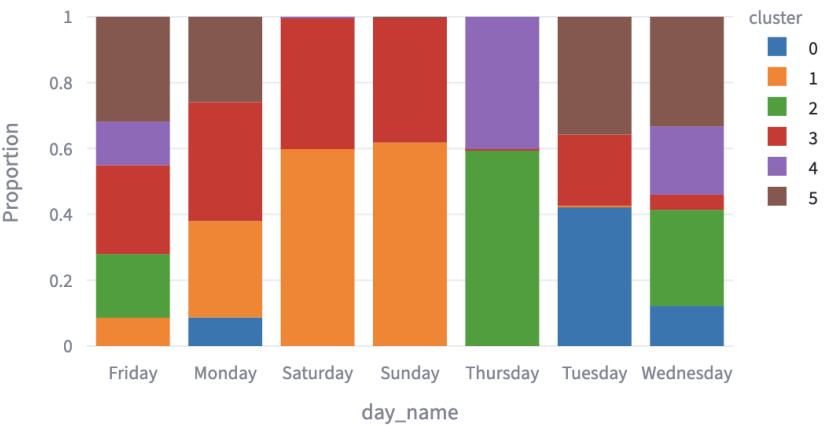


Figure 24: Daily cluster distribution analysis revealing day-specific operational patterns

immediate classification of new sensor measurements against previously identified operational patterns. The interface provides smart preset values pre-populated with cluster centroids for realistic starting points, dynamic validation with real-time bounds checking based on historical data ranges, and a comprehensive confidence assessment derived from distance calculations to all discovered clusters.

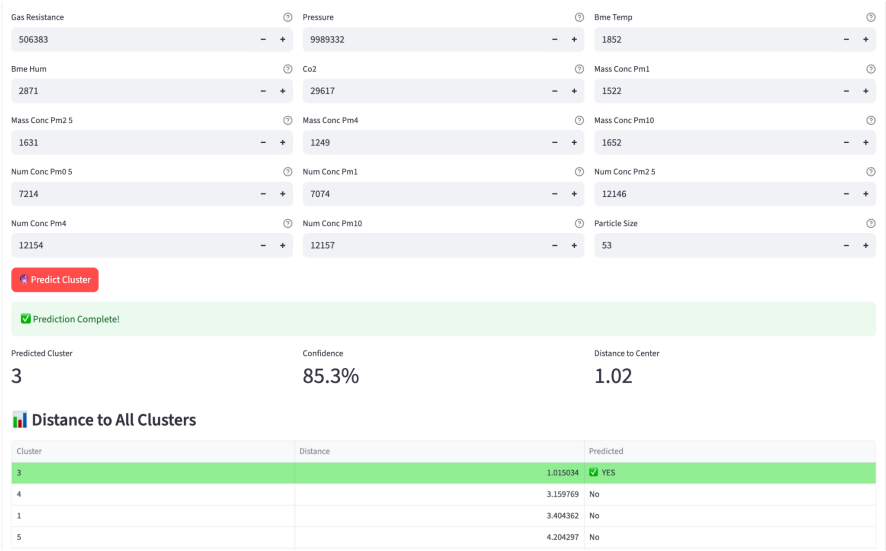


Figure 25: Real-time prediction interface showing immediate classification of new sensor measurements

The real-time processing pipeline enables instant classification through input validation against historical data bounds, feature scaling using the same StandardScaler transformation from training, K-means prediction that calculates distances to all cluster centroids simultaneously, and confidence assessment that derives certainty percentages based on distance ratios. The system typically

responds in under one second, delivering predicted cluster assignments, confidence scores, complete distance profiles to all operational states, and ranked alternative possibilities for uncertain classifications.

7 Sensor Cross-Analysis

Here, we look at how readings from a sensor correlate with readings from another sensor. Upon blindly looking at the raw data's correlation heatmap, we see some anomaly: the SCD30 sensor and BME680 both measure humidity and temperature, and we would expect them to have high correlation. While we see a correlation score of +0.97 between humidity recorded by the two sensors, we find that there is only a +0.34 correlation score between temperatures recorded by the sensors, as shown in Figure 26.

Unsatisfied with the temperature correlation score, we looked at the scatter plot between the temperatures recorded by the sensors, as shown in Figure 27. We observe a clear linear cluster in the middle, which likely corresponds to the 'normal' sensor readings. We have an explanation for the points that deviate from this linear relationship: the flat line at `temperature_bme` between 32.5 and 35 is possibly due to stuck readings or overheating of the BME sensor. This case is less frequent, with only 18.77% of the total records exhibiting this anomaly.

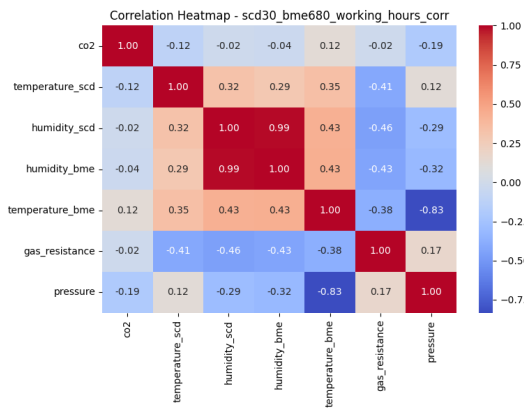


Figure 26: Correlation heatmap between SCD30 and BME680 readings

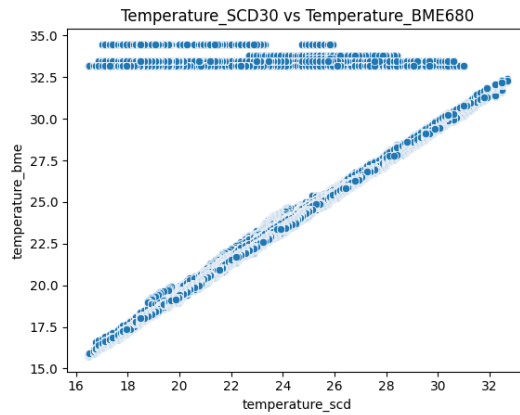


Figure 27: Scatterplot of temperature readings from SCD30 and BME680

This high humidity correlation and the underlying linear relationship between the temperature readings suggest redundancy between the sensors. Hence, the only new information recorded by the SCD30 sensor is the CO₂ readings. So we looked if there exists a relationship between CO₂ from SCD30 and the sensor readings from BME680 (temperature, humidity, gas_resistance and pressure). The BME680 sensor records gas_resistance by measuring volatile organic compound (VOC), as VOC has a negative relationship with gas_resistance [3]. Additionally, [4] state that VOCs share the same temporal pattern as CO₂, which meant that gas_resistance and CO₂ have a negative relationship. Hence, we tried to create various models that could predict the CO₂ readings based on gas_resistance readings. An example of such negative relationship between CO₂ and gas_resistance can be seen in Figure 28 which was found during our Data Visualization and Analysis phase Section 3.

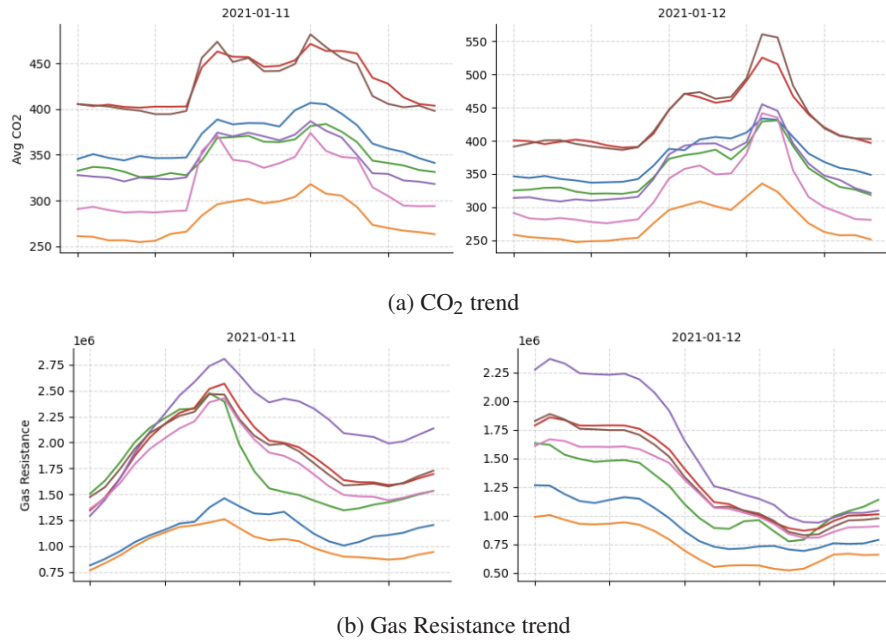


Figure 28: Negative correlation between CO₂ and Gas Resistance

Based on the humidity and temperature redundancy and relation between CO₂ and gas_resistance, it seems that use of SCD30 sensor can be avoided. To this end, we tried creating a model that estimates the value of CO₂ readings based on readings from BME680 sensor. We tried a lot of strategies; different models and different training dataset; which are discussed below:

It is important to mention that in all the strategies we experimented with, the training data used is a subset of the whole data; we used only the records which have timestamp values in the working hours (9AM to 6PM) of the process days, which we obtained from section ???. This subset was selected because the non-process days (aka baseline days) do not show much activity and the readings are stagnant; the sensor reads the same static value. Also, in any strategy, the dataset is split to a train_split and a test_split in the ratio 80:20. Additionally, the 'model A' refers to the model trained with features (X) as time related features and gas_resistance values only and 'model B' refers to the model trained with features (X) as time related features and all BME readings, i.e., temperature_bme, humidity_bme, gas_resistance and pressure. Additionally, please note that when performance of a model is discussed, there are 2 accuracy values mentioned, the first one is for $\pm 5\%$ tolerance and the second one is for $\pm 10\%$ tolerance.

First, we created a Neural Network regression model trained on March and June 2022 data only. Initially we trained model A on this data. The model performed good on the test_split with an accuracy of 83.79% and 96.12% accuracy. Then, we also trained model B on the same data. This model performed even better on the test_split with a 96.67% and 99.72% accuracy.

However, on completely new data for a different month, both these models performed badly, (e.g., testing on April 2021, model B performed with an accuracy of just 29.11% and 50.09%. Model A performed even worse). So far, we saw that model B with more features is performing better than model A, and both these models, perform well on the test_split but perform bad on unseen data. So we focused on metrics on unseen data from then on.

Moving forward, we then considered model B more often than model A. We trained model B on the data of 2021 and tried to test it on 2022 data, and vice-versa. Here, when trained on 2021 data and tested on 2022 data performed with an accuracy of 24.47% and 45.36% and in the vice-versa case, the model performed with an accuracy of 18.07% and 33.57%. Given this bad performance of model B, we wondered how model A would perform in this scenario.

Surprisingly, model A performed better than model B, (e.g., while trained on 2022 data and tested on 2021 data, it gave an accuracy of 29.07% and 50.09%). Here, we learned that the different years had different semantics and the model trained on one year could not perform well on data of another year. We also started to doubt if our model was overfitting. Also, we realized that neither model A nor model B is superior to the other, and the choice is up to the user.

Then, to counter overfitting, we tried changing the architecture of the model. Previous architecture was a basic 2 layer module, with no dropout layer. So we added a dropout layer with dropout probability 0.3 and we added one more hidden layer with dropout probability 0.2, making the model denser. We chose the dropout probabilities to be smaller; not more than 0.5; (0.3 and 0.2) because while we wanted the model to not overfit, we did not want it to generalize too much either. The model architectures we finalized on is shown in Figure 29. Then we trained model A on Feb 2022 data and tested on Feb 2021 data. The model with changed and better architecture still performed bad, with an accuracy of 13.57% and 29.85%.

```
# Define the optimized model
model = Sequential([
    Dense(64, activation='relu', input_shape=(X.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(64, activation='relu'),
    BatchNormalization(),
    Dropout(0.3),

    Dense(32, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(1) # Output layer for regression
])
```

Figure 29: CO₂ Prediction Model Architecture

To this end, we created a dataset containing records from both 2021 and 2022. For training, records with dates between 01 and 20 of all the months were selected. The remainder (days between 21 and end of month) was used as unseen data during testing. We trained model A on this dataset and it showed an accuracy of 47.95% and 75.92%. This shed light that the model trained on both years would perform better.

We also tried incremental/online learning strategy, where the model trains on each month's data chunk, and updates continuously as new month's data is available. We tried this strategy using model A and data from Jan to Apr 2021. But this strategy also performed poorly with accuracies of 14.54%, 48.93%, 18.56% and 43.84% on Jan, Feb, Mar and Apr respectively under

$\pm 5\%$ tolerance.

So we finally trained model A and model B on the whole dataset, and it performed considerably well on the test_split with accuracies of 73.05% and 76.19% within $\pm 5\%$ tolerance and 94.17% and 95.91% within $\pm 10\%$ tolerance. But this model could not be validated on unseen data (e.g., 2023 data) due to limited access to the IMF lab data. The training loss vs validation loss of model A and model B can be seen in Figure 30.

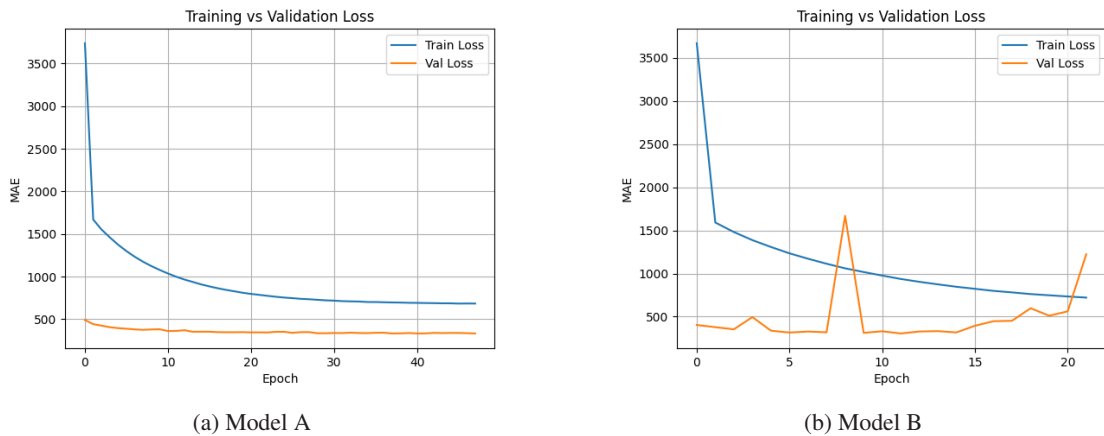


Figure 30: Training loss vs Validation loss

In our framework, the user can use either of these pretrained models A and B to predict CO₂ values. Given a new dataset of a completely new setting, the user also has the option to train their own model (model A or model B) and can choose the date range of the dataset they want to train on and can also select the node_id values; as shown in Figure 31, as they can be different for different dataset. The user can also mention the number of epochs they want the model to train. After training, the user can also download the scaler.pkl and model.keras file for future use. The most recently trained model and scaler is stored by the framework for each model type and can be used by the user to test its performance.

Figure 31 shows a web interface for model training. The main heading is 'Select date range of data you want to use to train the model'. Below this, there are input fields for 'Start Date' (2022/11/18) and 'End Date' (2022/12/18). A 'Node Selection' section shows a list of selected node IDs: 9, 12, 10, 6, 8, 11, and 7. The 'Choose how you want to train your model' section shows a dropdown menu for 'Select Model type' with 'Model A (Gas Resistance + Time)' selected. Below this, there is an input field for 'Enter Number of Epochs you want to train the model for' with the value 15. A 'Create model' button is visible at the bottom left. At the top right, there are 'Stop' and 'Dej' buttons.

Figure 31: Example of Model training by user

The user can test the model in 3 different ways:

1. By providing the features (X) of a single row. Example can be seen in Figure 32.
2. By uploading a CSV in the correct format. Example can be seen in Figure 33. The user can also download the file with predicted CO₂ readings.
3. By choosing a date in the database and a node_id and comparing between the predicted and actual CO₂ values. Example shown in Figure 34.

Entering Details for a Single Row ▼ Depl

 **Provide Input for Prediction**

Select Node ID

6 ▼

 **Enter Timestamp and Node Info**

Select Date

2021/11/15

Enter Hour (0-23)

10

Enter gas resistance.

450424

 Predict CO₂ Level

 Predicted CO₂ Level: 375.89 ppm

Figure 32: Example of Predicting CO₂ by providing details of a single row

 **Upload CSV for Batch Prediction**

Upload a CSV file with a 'timestamp', 'gas_resistance', and 'node_id' column

 Drag and drop file here
Limit 200MB per file • CSV

 example.csv 448.0B ×

 Predictions completed!

	month_cos	week_number_sin	week_number_cos	node_6	node_7	node_8	node_9	node_10	node_11	node_12	predicted_co2
0	002	1	-0.1205	0.9927	☒	☐	☐	☐	☐	☐	306.9866
1	002	1	-0.1205	0.9927	☐	☐	☒	☐	☐	☐	552.7454
2	002	1	-0.1205	0.9927	☒	☐	☐	☐	☐	☐	439.1749
3	002	1	-0.1205	0.9927	☐	☐	☐	☐	☒	☐	535.4441
4	002	1	-0.1205	0.9927	☐	☒	☐	☐	☐	☐	412.2923

 Download CSV with Predictions

Figure 33: Example of Predicting CO₂ by uploading a CSV file

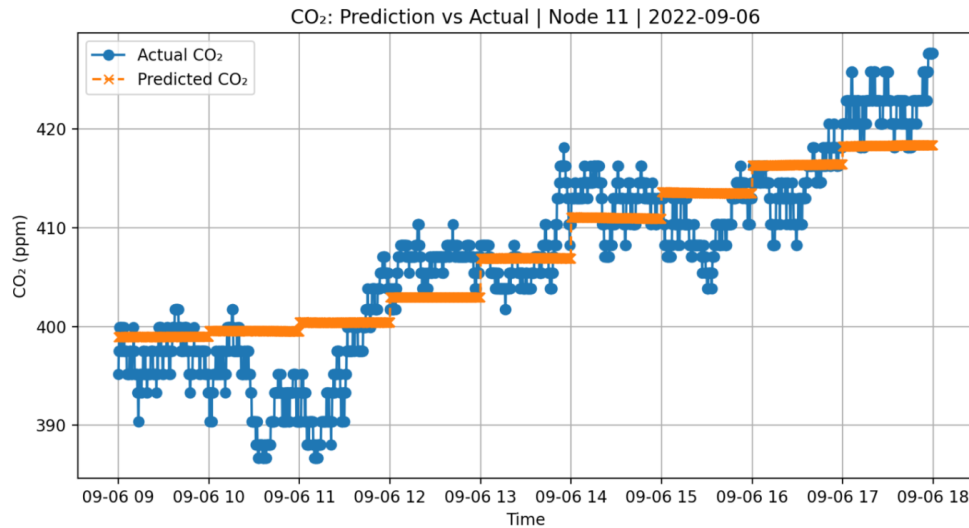


Figure 34: Example of Predicting CO₂ for a single day and comparing against the actual CO₂ values

8 Node Analysis

This section is to understand the behavior of the nodes, detect anomalies, assess any redundancies, and view cluster patterns of the data recorded by the nodes. The nodes represent different locations within the room where the sensors recorded the data. Instead of focusing on individual sensor measurements as done in previous sections, this page enables cross-node comparisons so that we can identify any operational anomalies (when nodes are exhibiting unusual behavior for any sensor compared to the other nodes), behavioral clustering (to group nodes with similar patterns, to suggest similar performance and give validity to the data), data redundancies (find nodes that are reporting nearly identical readings, to potentially remove the node to save costs and avoid unnecessary duplication of data) and sensor drift or degradation (to spot inconsistencies over time to indicate any calibration issues or sensor failures). The goal of this page is to track deviations, similarities, and patterns at the node level over time. Figure 35 below shows the layout of the nodes within the room for reference.

8.1 Trend Viewer

This feature in the framework allows for exploring sensor features by the node, date and plot type. The page has 4 different plot types: line, bar, heatmap and trend over multiple days. Line graphs compare raw time-series for selected nodes. The bar graph shows the hourly aggregated values for the selected data. The heatmap helps to visualize hourly intensities across nodes and the trend over days is to track the behavior of a single node over multiple days.

8.1.1 Line Plot

Figure 36 shows the line plot graph of all of the nodes on 18 December 2020 specifically for humidity. Each line represents data from a specific node, allowing for direct comparing of humidity values in the room. Most of the values recorded follow a similar trend in this graph and are consistent, suggesting that the environment was fairly uniform in the humidity distribution.

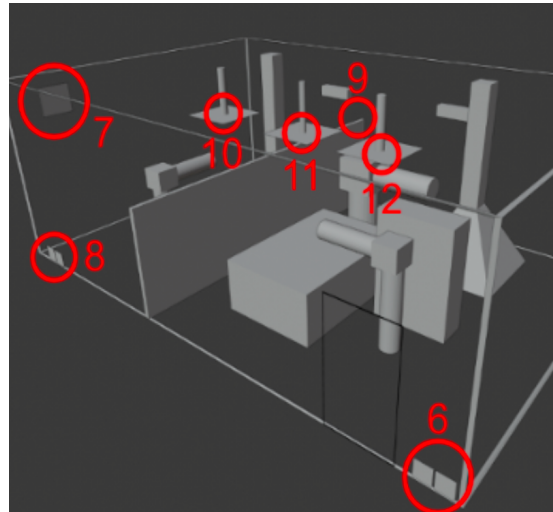


Figure 35: Node Layout

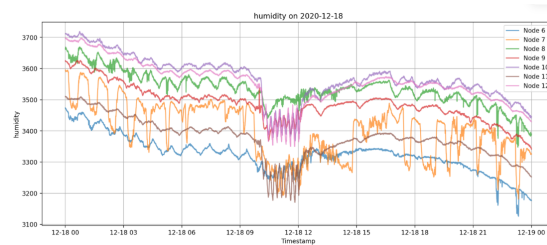


Figure 36: Line Graph, Humidity, All Nodes, 18-12-2020

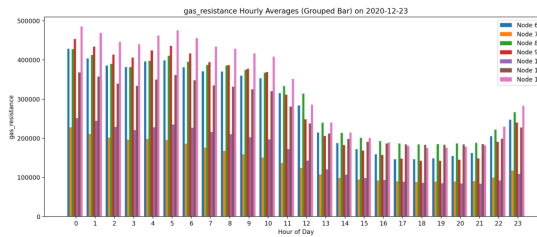


Figure 37: Bar Plot, Gas Resistance, All Nodes, 23-12-2020

The small differences in data recorded can be attributed to the differences in node placement. In this particular graph, only node 7's recordings are slightly more volatile, fluctuating up and down at a greater rate than the other nodes. However, the range of values still matches the other node values. This suggests that it is not malfunctioning but may be located in an area with more dynamic airflow. This helps to identify micro-environmental differences that can be further analyzed or investigated.

8.1.2 Bar Plot

The bar plot graph in Figure 37 shows a bar plot depicting the hourly gas resistance readings recorded by the sensor nodes on 23 December 2020. Each bar represents the value recorded by the node at that time. Overall, the trend is the same across all nodes, with some nodes recording a lower value than other but the shape remaining consistent throughout the day. The consistent changes across nodes suggests that fluctuations are uniformly distributed and most likely influenced by the same factors, without any issues or concerns. This type of plot is helpful in identifying any outlier behavior. However, in this instance, no anomalies are present, reinforcing confidence in the data quality.

8.2 Anomaly Detection

This section is designed to identify unusual or abnormal sensor node behavior, based on deviation z-score. This can help to find faulty sensors, any unexpected environmental disturbances, nodes diverging from group patterns due to reasons like calibration issues and specific days where a specific node might not be aligned with the values recorded by the other nodes. The anomalies are detected using the Z-score analysis, which helps to identify on an hourly basis when a node's readings are significantly different from the group average. The user can select the sensor feature, any date within the database and a Z-score threshold. The framework will calculate when the z-score for a node at any hour in that day exceeds the threshold compared to the other nodes. There are 3 graphs generated - Outlier Heatmap, Anomaly Count per Node and Deviation Heatmap. The final table - All Detected Anomalies - collates every anomaly logged in the session and can be downloaded and exported.

The heatmaps in Figure 38 and 39 visualize anomalies detected across sensors for chosen features and dates. Each cell indicates whether a node's reading was anomalous compared to the data recorded by the other nodes at that hour based on the Z-score analysis. The dark red cell represents an anomalous reading.

Evidently from the heatmaps in Figures 38 and 39, node 7 recorded many data points considered anomalous on both 23 and 25 Jan 2021. Additionally, when collating the days when a node showed anomalous data in the month of January for the features humidity and temperature, the following

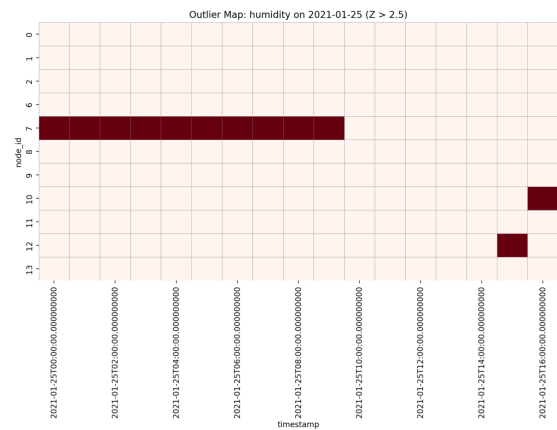


Figure 38: Outlier Map, humidity, 25-01-2021 Note: Ignore nodes beyond 6-12

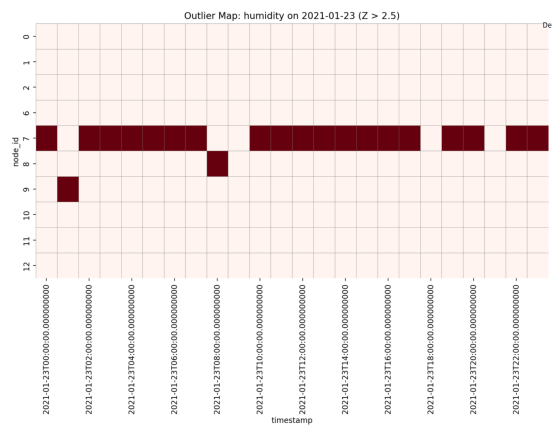


Figure 39: Outlier Map, humidity 23-01-2021 Note: Ignore nodes beyond 6-12

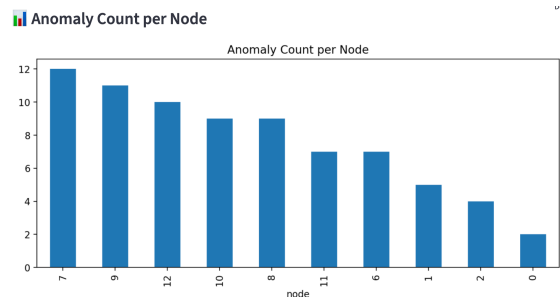


Figure 40: Collective Anomaly Count for January 2020 Note: Ignore nodes beyond 6-12

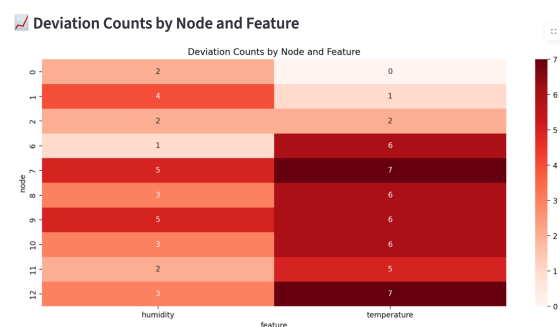


Figure 41: Deviation Counts for January 2019 Note: Ignore nodes beyond 6-12

graphs were generated: Figures 40 and 41. This suggests that node 7 is collecting the most anomalous data points in the month, followed by node 9, then 12 and so on. The deviation counts table shows the number of deviations per node for the relevant feature in the month of January 2019. These results are worth looking into.

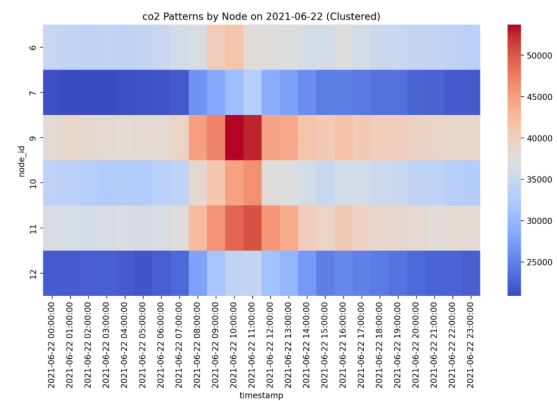
The patterns suggests that node 7 consistently recorded anomalous data points throughout the month. Results like this may be due to issues like sensor mis-calibration, hardware degradations or due to a valid reason like micro-environmental changes due to being near a vent or door etc. Upon reviewing the location of the node in the room, it is located away from other nodes at the end of the room, which may cause it to experience a unique micro-environment, in turn causing the data results. Either way, it is worth investigating further. Nodes with consistently high deviation counts across multiple features would need further investigation or recalibration.

The findings highlight the value of anomaly detection in identifying early warning signs of faulty or diverging nodes, which is very critical in maintaining data integrity in the long-term in environmental monitoring networks.

8.3 Clustering Analysis

This section explores how sensor nodes group together based on their hourly behavior patterns for the selected features. Using K-Means, nodes with similar feature trends across the day get assigned to the same cluster to help identify groups of nodes behaving similarly, due to spatial proximity or shared environmental conditions and consistent groupings across multiple days, which could confirm spatial zoning assumptions.

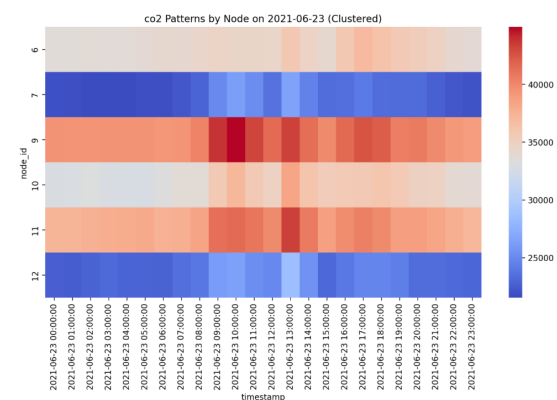
When looking at the clustering analysis of CO₂ across different days in June 2021 in Figures 42 and 43, we can see which nodes cluster together showing similar values. As is clear in figures



Cluster Assignments

	node_id	cluster
0	9	0
1	11	0
2	7	1
3	12	1
4	6	2
5	10	2

Figure 42: CO2 Clustered, 22-06-2021



Cluster Assignments

	node_id	cluster
0	9	0
1	11	0
2	7	1
3	12	1
4	6	2
5	10	2

Figure 43: CO2 Clustered, 23-06-2021

42 and 43 , for CO₂, nodes 9 and 11, 7 and 12, and 6 and 10 are clustered together, showing similar values. This trend repeats across different days in the month. These results could be attributed to the proximity of the nodes within the room, being placed in zones affected by the same ventilation, or layout, etc. However, when expanded across multiple months, the cluster groupings do not remain consistent for this facility and room, indicating that there are no node pairs in this specific IWSN.

If the nodes are consistently clustering together across features for months, it might indicate redundancy and only keeping 1 node may be sufficient for future models and tasks. The patterns can help to make decisions on sensor deployment, maintenance and more for further analysis. On the other hand, having a confirmed node pair could allow the user to use the other node pair to confirm and verify the data collected in case there are issues or errors to check if there was a temporary outlier or if there is a more serious issue with the node or sensor that needs to be fixed or replaced.

9 Conclusion

In this project, we set out to build a robust, adaptable, and intelligent system to process sensor data from Industrial Wireless Sensor Networks. By integrating data from multiple sensors we developed a flexible pipeline that spans from raw data ingestion and visualization to advanced unsupervised learning for operational state detection.

Our irregularity and disruption analysis highlighted the real-world challenges of sensor deployments, including data gaps, faulty readings, and synchronization issues. The integration of simple machine learning models showed that even lightweight methods, when properly applied, can effectively detect anomalies and trigger timely maintenance actions. A key milestone was the development of the intelligent sensor state analysis System, built on unsupervised clustering techniques. This enabled automatic detection of distinct operational states and uncovered meaningful temporal patterns without the need for labeled data. The system's ability to distinguish between process days and baseline days solely through sensor features underscores the power of data-driven insights in industrial monitoring.

Finally, node-level analysis, including anomaly detection and clustering, uncovered spatial patterns and behavioral consistency across sensor nodes. This facilitates better sensor placement, proactive maintenance planning, and strategic redundancy management. In summary, the system offers a useful and user-friendly framework for engineers, researchers, and decision-makers to interpret and act on complex sensor data. It bridges the gap between raw data-points and actionable insights. As industrial environments continue to evolve, adaptive systems like this will become increasingly useful for ensuring efficiency, reliability, and sustainability.

References

- [1] Adrien Treuille, Amanda Kelly, Thiago Teixeira, et al. Streamlit: The fastest way to build and share data apps. <https://streamlit.io>, 2019. Accessed: 2025-07-16.
- [2] Yue Zhao, Zain Nasrullah, and Zheng Li. Pyod: A python toolbox for scalable outlier detection. *Journal of Machine Learning Research*, 20(96):1–7, 2019.
- [3] Luis Llamas. Environmental quality sensor with arduino and bme680. <https://www.luisllamas.es/en/environmental-quality-sensor-arduino-bme680/>, 2020. Accessed: 2025-07-14.
- [4] Betty Molinier, Caleb Arata, Erin F Katz, David M Lunderberg, Jennifer Ofodile, Brett C Singer, William W Nazaroff, and Allen H Goldstein. Bedroom concentrations and emissions of volatile organic compounds during sleep. *Environmental Science & Technology*, 58(18):7958–7967, 2024.